

TODO SUMMARY ASSISTANT

Project Report

1. Introduction

Todo Summary Assistant is a web application that allows users to create and manage todo items. The interface provides options to add tasks, mark them complete, edit or delete them, and summarize pending todos with an option to send the summary to Slack.

2. Objectives

Develop and deploy a todo-management web application.

Automate testing, container image building, publishing, and deployment using Jenkins.

Package application components using Docker and Docker Compose.

Monitor application and server metrics using Prometheus and Grafana.

3. Tools and Technologies

Jenkins, GitHub, Maven, Docker, Docker Compose, Docker Hub, AWS, Prometheus, Grafana, Node Exporter, and cAdvisor.

4. Implementation

The source code is maintained in a GitHub repository.

Jenkins runs the Maven verification tests for the backend.

Docker images are built for the frontend and backend and pushed to Docker Hub.

The application is deployed to AWS servers using Docker Compose.

Application configuration values are retrieved from AWS Systems Manager Parameter Store.

Prometheus is configured to scrape metrics from the backend and server/container monitoring targets.

Grafana dashboards visualize CPU and memory usage for the application and monitoring servers.

5. Testing and Results

The Jenkins pipeline completed successfully in the provided build log.

The Maven test stage passed with one test and no failures or errors.

Frontend and backend Docker images were built and pushed successfully.

The application health check returned the Todo Summary Assistant webpage.

Prometheus showed the app-node, backend, cAdvisor, and monitoring-node targets as UP in the supplied screenshot.

Grafana displayed CPU and memory usage graphs for the monitored servers.

6. Database Verification

Database connectivity and persistence have not yet been independently confirmed in the evidence reviewed. Further testing should verify that the configured database is available, that the backend can connect to it, and that todo items remain stored after being created and retrieved.

7. Conclusion

The project demonstrates an automated CI/CD workflow and a basic monitoring setup for a containerized web application. The Jenkins deployment pipeline completed successfully, and Prometheus and Grafana displayed monitoring data. Database connectivity and data persistence remain to be verified as a final validation step.

8. Future Enhancements

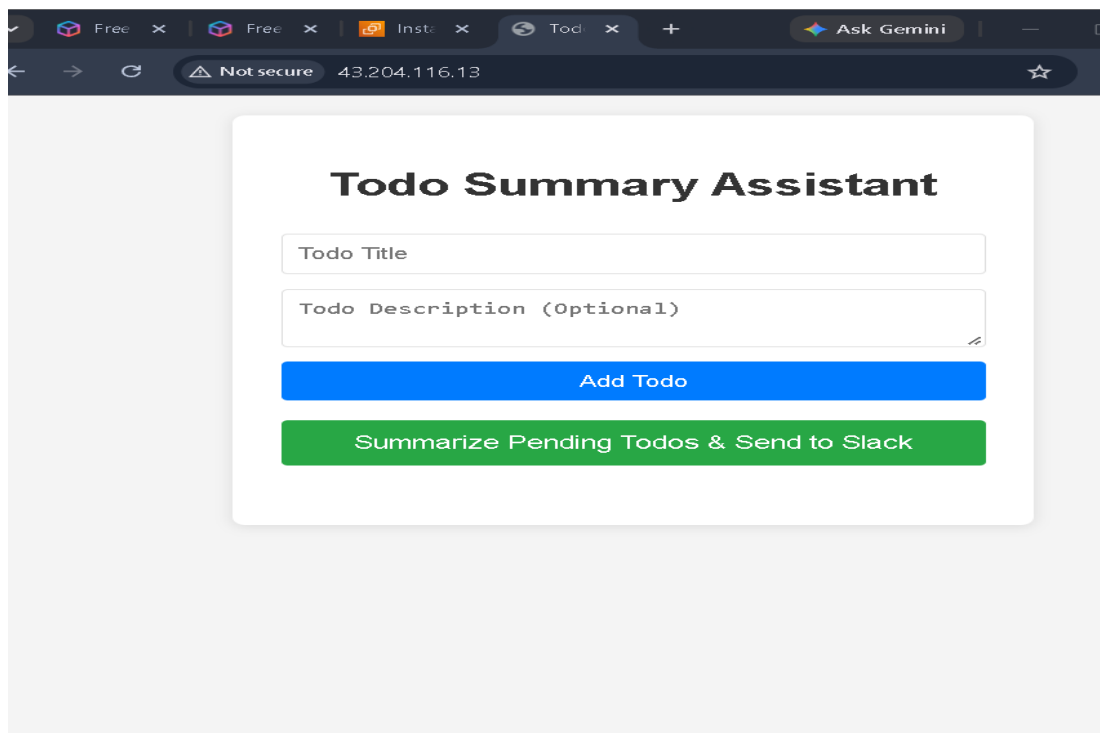
Configure alerts for high CPU or memory usage and unavailable monitoring targets.

Create additional Grafana panels for backend request metrics and container health.

Complete database connectivity and data-persistence testing.

Restrict monitoring dashboard access to authorized users.

SnapShots



After deploying the application to EC2 instance through CI/CD pipeline

Todo Summary Assistant

Todo added successfully!

Todo Title

Todo Description (Optional)

Add Todo

Summarize Pending Todos & Send to Slack

project-dmeo

description

Mark Complete

Edit

Delete

Monitoring snapshots

Select scrape pool	Filter by target health	Filter by endpoint or label	
app-node	Show config	1 / 1 up	
Endpoint	Labels	Last scrape	State
http://172.31.14.32:9100/metrics	instance="172.31.14.32:9100" job="app-node"	15.054s ago 15ms	UP
backend	Show config	1 / 1 up	
Endpoint	Labels	Last scrape	State
http://172.31.14.32:8080/actuator/prometheus	instance="172.31.14.32:8080" job="backend"	7.11s ago 12ms	UP
cadvisor	Show config	1 / 1 up	
Endpoint	Labels	Last scrape	State
http://172.31.14.32:8081/metrics	instance="172.31.14.32:8081"	2.273s ago	UP

